



Indexing in DBMS

Agenda

- ▶ **Definition, Importance and Purpose**
- ▶ **Index Data Structures**
- ▶ **Types of Indexes**
- ▶ **Indexing in SQL**
- ▶ **Summary**



Indexing in DBMS

Definition, Importance & Purpose

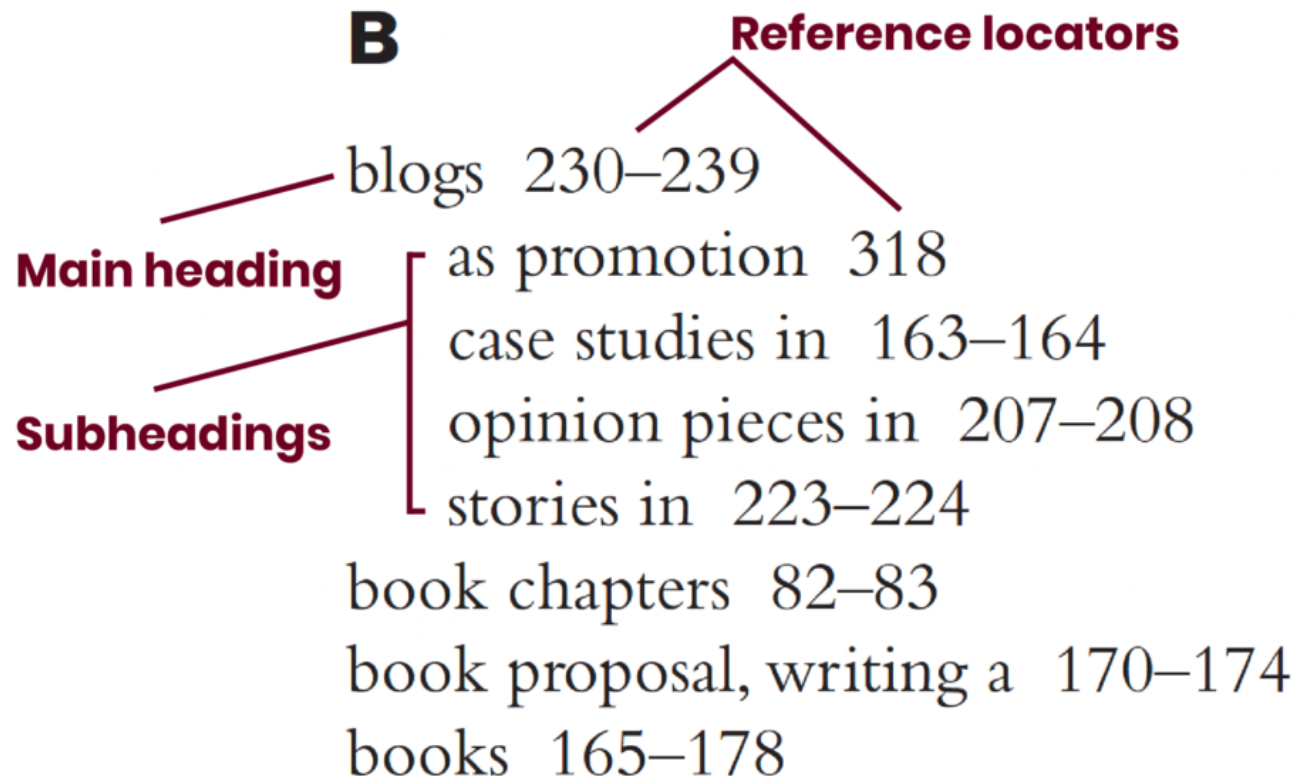
Indexing

- ▶ Indexing in DBMS is used to speed up data retrieval by minimizing disk scans.
- ▶ Indexing is the process of creating a data structure that allows the database to quickly locate and retrieve rows without scanning the entire table.

Indexing – Book Index

- Think of an index like the index at the back of a book: instead of scanning every page to find a topic, you jump directly to the page number listed in the index.

Indexing – Book Index

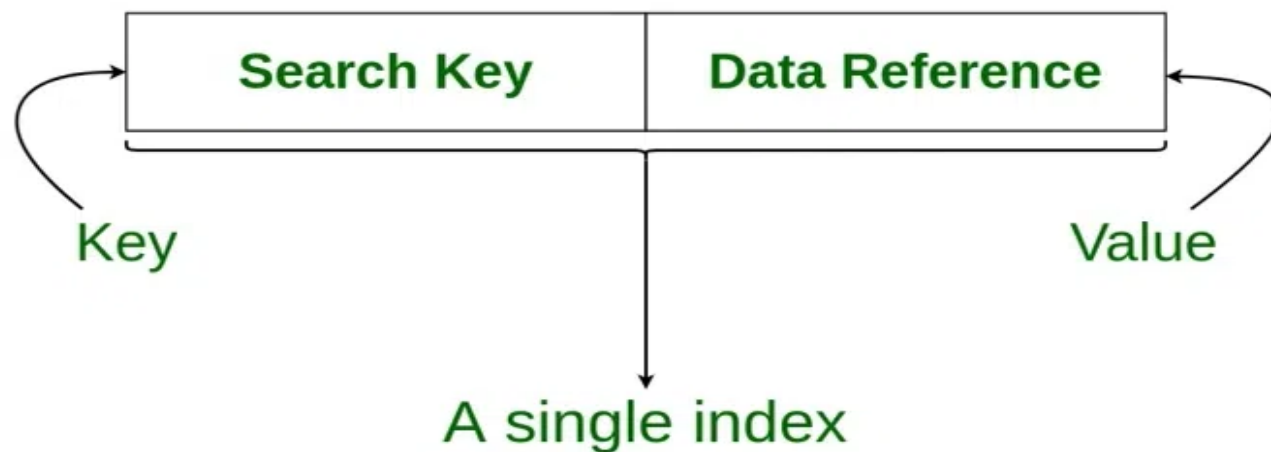


Indexing – How it works

- ▶ When an index is created, it stores sorted key values and pointers to actual data rows.
- ▶ This reduces the number of disk accesses, improving performance especially on large datasets.

Index Structure

Structure of an Index in Database



Indexing - Sample

INDEX FILE	
SEMESTER	INDEX ADDRESS
1	
2	
3	
4	
5	

Data Blocks in Memory					
	100	Joseph	Alaledon Township	20	200
	101				
	110	Allen	Fraser Township	20	200
	111				
	120	Chris	Clinton Township	21	200
	121				
	200	Patty	Troy	22	205
	201				
	210	Jack	Fraser Township	21	202
	211				
	300				

Indexing – Factors

- ▶ **Access Types:** This refers to the type of access such as value-based search, range access, etc.
- ▶ **Access Time:** It refers to the time needed to find a particular data element or set of elements.
- ▶ **Insertion Time:** It refers to the time taken to find the appropriate space and insert new data.
- ▶ **Deletion Time:** Time taken to find an item and delete it as well as update the index structure.
- ▶ **Space Overhead:** It refers to the additional space required by the index.

Advantages

- ▶ **Faster Queries**
- ▶ **Efficient Access**
- ▶ **Improved Sorting**
- ▶ **Consistent Performance**
- ▶ **Data Integrity**

Disadvantages

- ▶ **Increased Storage Space.**
- ▶ **Increased Maintenance Overhead.**
- ▶ **Slower Insert/Update Operations.**
- ▶ **Complexity in Choosing the Right Index.**



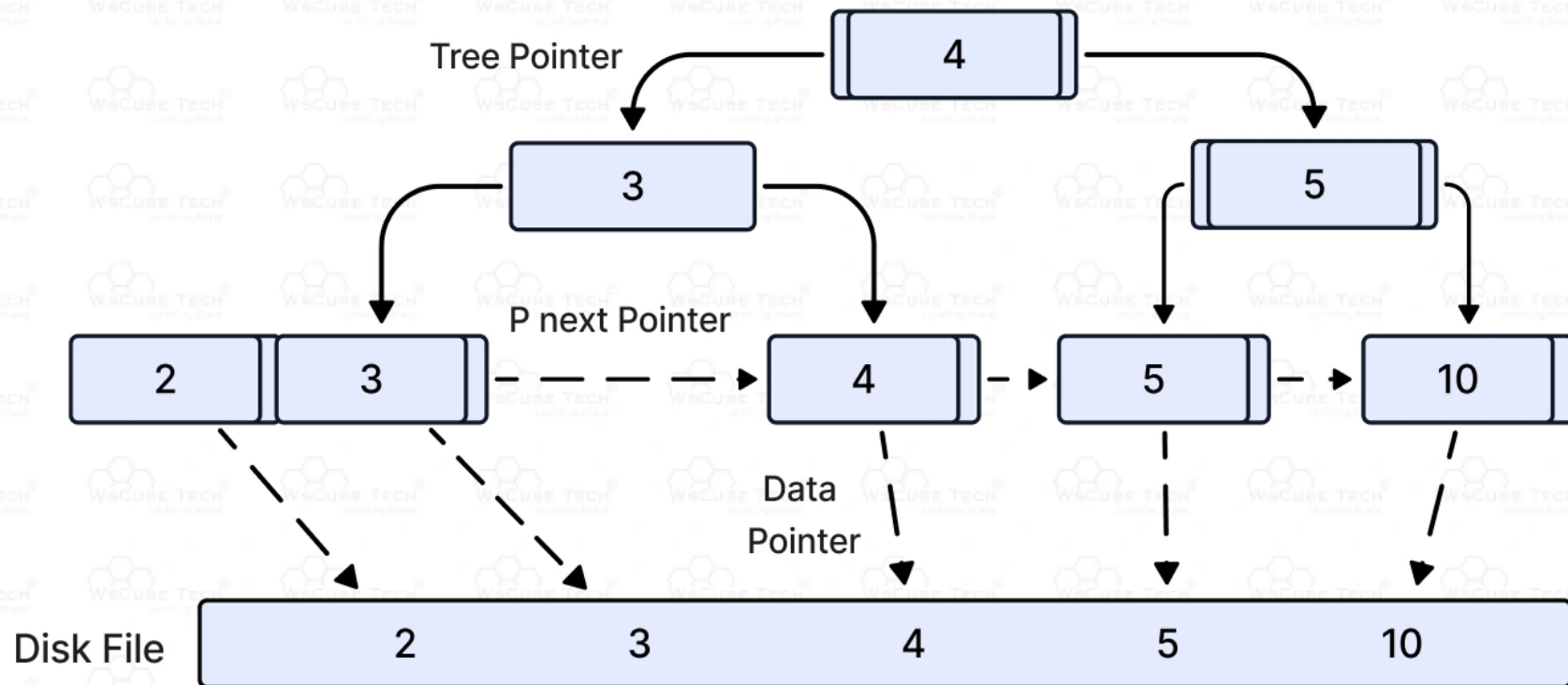
How does Indexes Work?

B+ Tree Index

- ▶ Most relational databases use a structure called B+ Tree for indexing.
- ▶ A B+ tree index is a hierarchical data structure used for efficient data retrieval by keeping data sorted and balanced.
- ▶ Fast retrieval, especially from disk.
- ▶ It is an extended version of the B Tree, where all actual data is stored only in the leaf nodes, while internal nodes contain only keys for navigation.

B+ Tree Index

B+ Tree



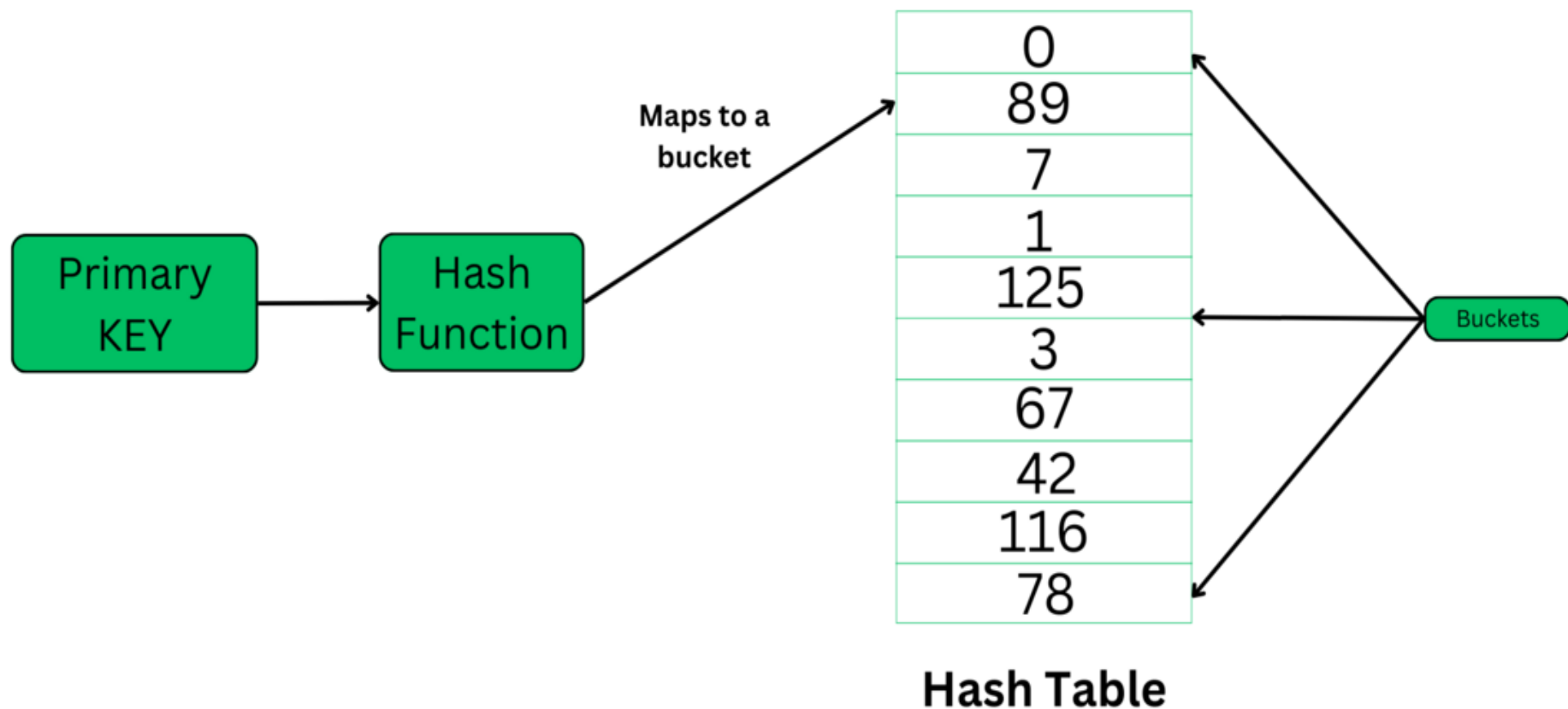
B+ Tree Index - Advantages

Fast search	Small height + sorted structure
Fast range queries	Leaf-level linked list
Good for ORDER BY	Index is sorted
Index-only scans	Data pointers/values at leaves
Predictable performance	All data at same depth
Efficient disk usage	High fan-out (many keys per node)
Fast sequential access	Leaf nodes linked
Easy maintenance	Self-balancing

Hash Index

- ▶ A hash index is a database structure that uses a hash function to create a fixed-size value, or "hash," from a variable-size input key to quickly locate data.
- ▶ There are 3 components in Hashing:
 - ▶ Hash Table – array of primary key and bucket
 - ▶ Bucket – memory location
 - ▶ Hash Function – generates bucket address

Hash Index



Hash Index - Advantages

- 1. Extremely Fast for Equality Searches**
- 2. Good for:**
 - 1. Primary key lookups**
 - 2. Searching a single record**
 - 3. Hash-joins**
 - 4. Hash-based in-memory lookups**
- 3. Simple data structure**
- 4. Efficient for huge databases**



Types of Indexing

TYPES OF INDICES

- ▶ **Primary Index**
- ▶ **Secondary Index**
- ▶ **Clustered Index**
- ▶ **Non-Clustered Index**
- ▶ **Dense Index**
- ▶ **Sparse Index**
- ▶ **Composite Index**

Primary Index

- ▶ A primary index is a type of ordered index built on the primary key of a table.
- ▶ There is one index entry per block (not per record).
- ▶ Since the data file is already sorted by the primary key, the index file also stores key values in the same sorted order.
- ▶ Because of this, the primary index is also called a sparse index.

Primary Index

Key (StudentID)	Block Address
--------------------	------------------

1001

B1

1005

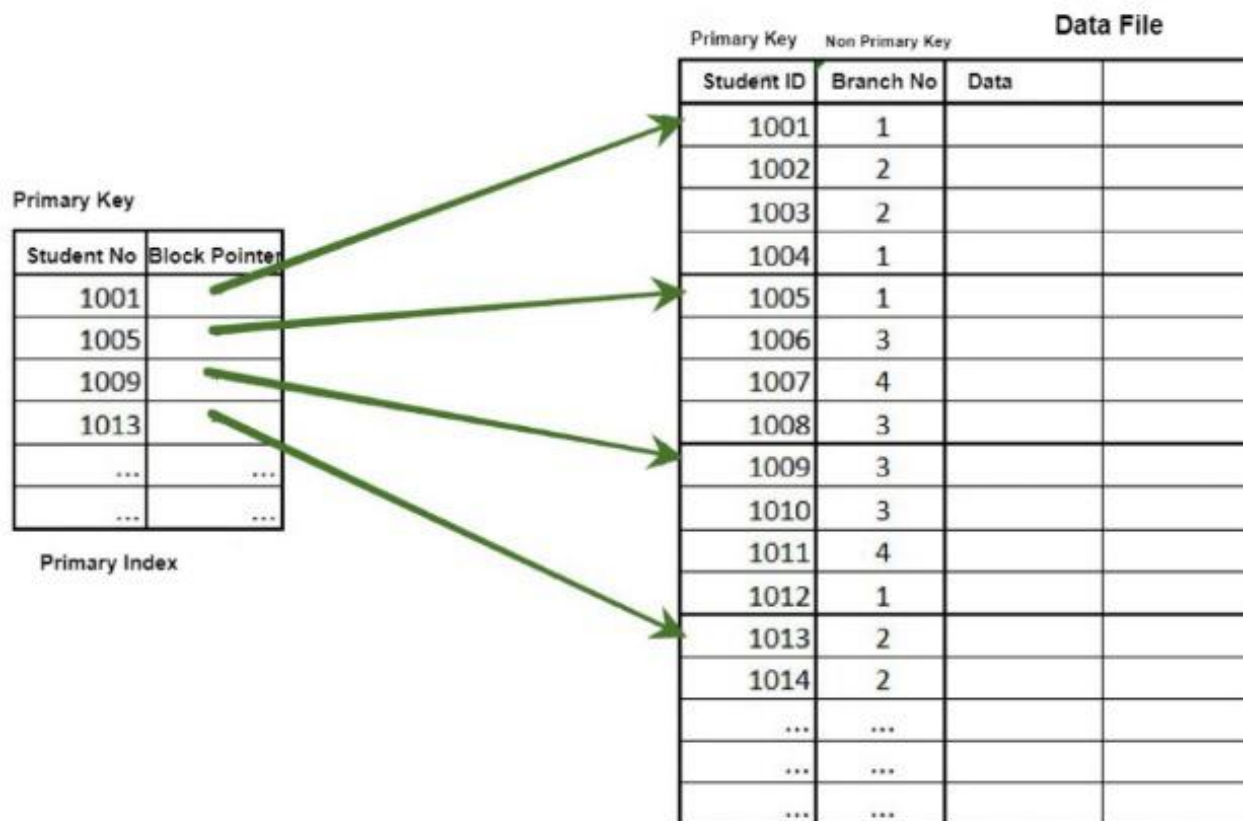
B2

1010

B3

Block	StudentID	Name
B1	1001	Ali
	1002	Sara
B2	1005	Ahmed
	1007	Maria
B3	1010	John
	1012	Fatima

Primary Index



Secondary Index

- ▶ A secondary index is an auxiliary index built on a non-key or non-ordering attribute.
- ▶ The data file is not sorted on this attribute.
- ▶ Therefore, the secondary index must contain one index entry per record (dense index).
- ▶ Even if the file is sorted on the primary key, searching on other attributes (like Name, Department, City, Salary) becomes slow. Secondary index solves this.

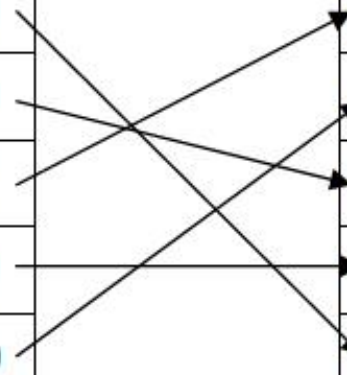
Secondary Index

Secondary Index

Search key	Address
Azad	0x2300
Priya	0x2100
Ram	0x1F00
Tom	0x2200
Vishal	0x2000

Data File

Address	<u>RegNo</u>	Name	Program	Age
0x1F00	14MS01	Ram	MS CS	21
0x2000	14MS10	<u>Vishal</u>	MS CS	22
0x2100	14MS29	<u>Priya</u>	MS CS	21
0x2200	14MS30	Tom	MS CS	21
0x2300	14MT59	Azad	<u>MTech</u>	23



Secondary Index

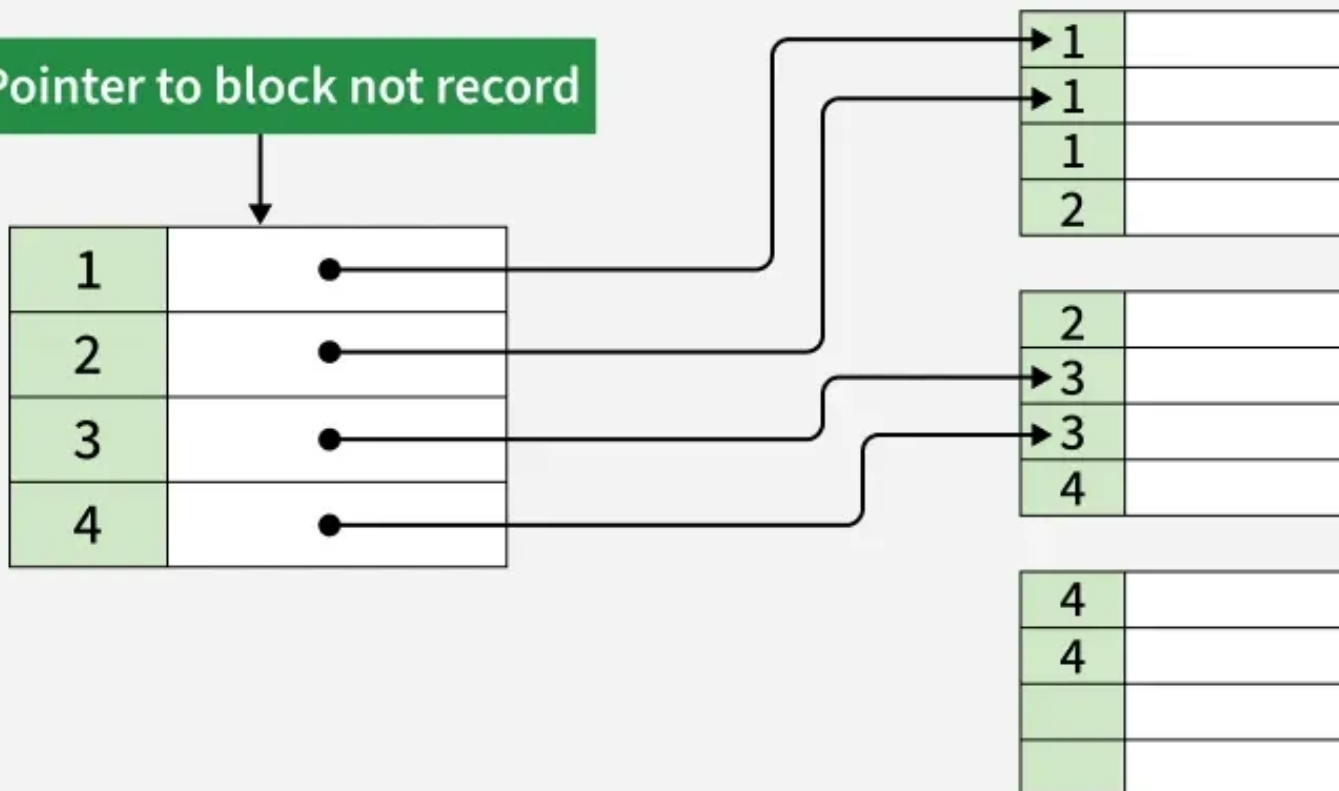
- ▶ **Helps speed up searches on non-key columns**
- ▶ **Might have duplicates (non-unique values)**
- ▶ **Requires additional storage**

Clustered Index

- ▶ A clustered index is an index that determines the physical ordering of the data file based on the index key.
- ▶ The table records are arranged (clustered) on disk according to that key
- ▶ There can be only ONE clustered index per table, because the data can be physically stored in only one order..

Clustered Index

Pointer to block not record



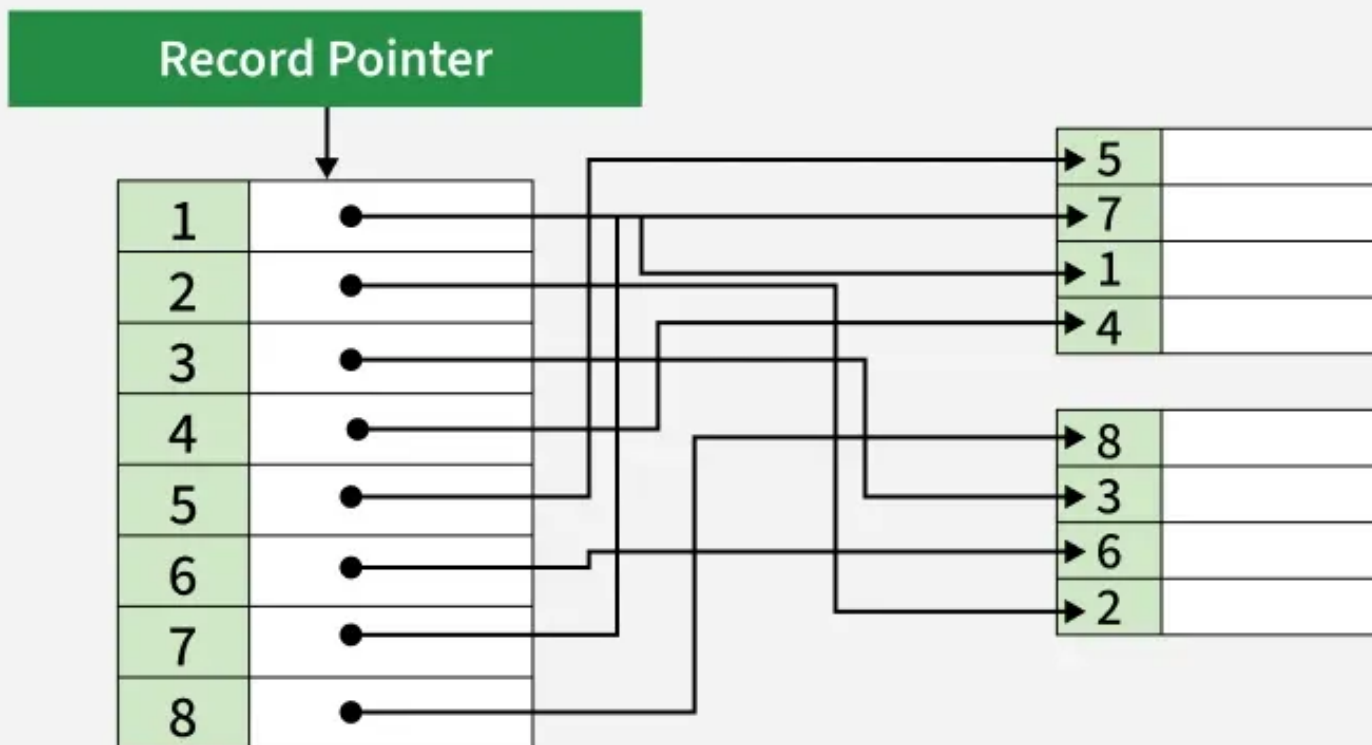
Clustered Index

- ▶ Faster for range queries (BETWEEN, >, <)
- ▶ Data and index are stored together
- ▶ Reduces I/O for sequential reads
- ▶ Slower for insert-heavy workloads (reordering needed)

Non-Clustered Index

- ▶ A Non-Clustered Index does not change the physical order of data.
- ▶ Instead, it creates a separate structure that stores the indexed column(s) along with pointers (row locators) to the actual rows in the table.
- ▶ This separation allows you to have multiple non-clustered indexes on a single table, making them useful for columns frequently used in search conditions or joins.

Non-Clustered Index

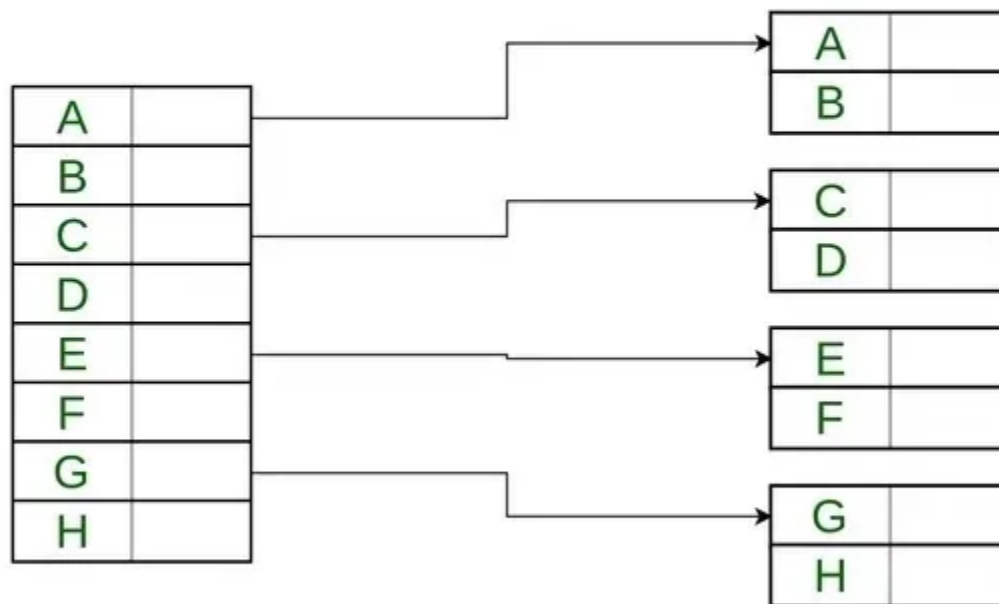


Sparse Index

- ▶ A sparse index is a database or data structure technique that creates an index entry for only a subset of a table's rows, rather than every row.
- ▶ This approach is used to save storage space and improve performance, especially in large datasets where many records share the same indexed value or are missing the indexed field.

Sparse Index

Sparse Index



Data File

Index Record

For very few
search value
in a Data File,

There is an
Index Record.

Hence the name
Sparse Index.

Sparse Index

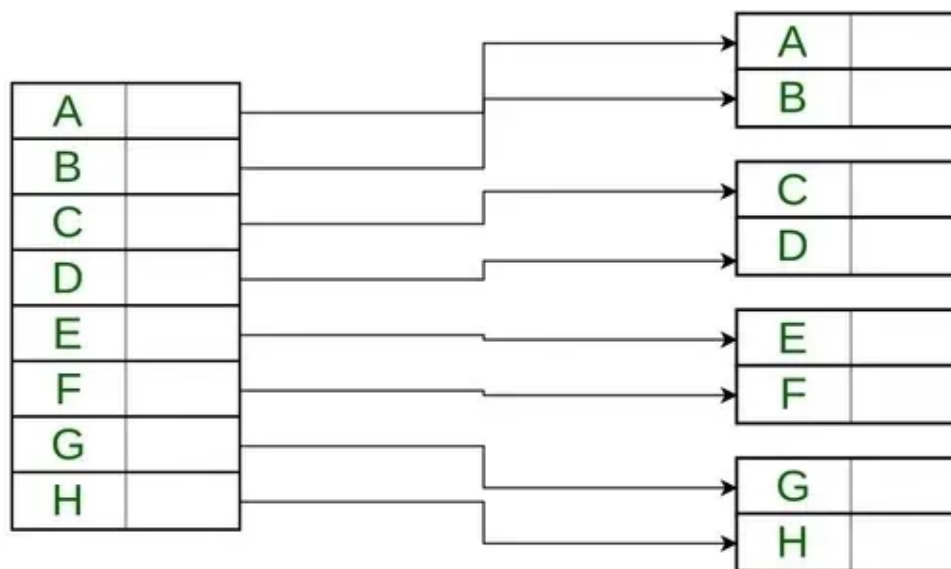
- ▶ **Less storage**
- ▶ **Slower than dense index**
- ▶ **Used with ordered / sequential files**
- ▶ **Usually tied to clustered index**

Dense Index

- ▶ It contains an index record for every search key value in the file.
- ▶ This will result in making searching faster.
- ▶ The total number of records in the index table and main table are the same.
- ▶ It will result in the requirement for more space to store the index of records itself.

Dense Index

Dense Index



Data File

Index Record

For every search value in a Data File,

There is an Index Record.

Hence the name **Dense Index**.

Dense Index

- ▶ **Fast Record Lookup:** No need to scan blocks; the record is directly found via index.
- ▶ **Efficient for Range Queries:** Since every key is indexed, traversing a range is easy.
- ▶ **Good for Frequently Queried Fields:** Especially useful in read-heavy applications.
- ▶ **Space Overhead:** Requires more memory and disk space to store the index.
- ▶ **Maintenance Cost:** Every insertion or deletion in the main table requires an update in the index.
- ▶ **Not Optimal for Huge Tables:** For very large tables, a multi-level indexing or B+ tree might be preferred.

Composite Index

- ▶ A **composite index** (also called a **multi-column index**) in a database is an index that is created on **two or more columns** of a table. Instead of indexing just a single column, it uses a **combination of columns** as the search key.

Composite Index

- ▶ Each entry in the index stores the combined values of the selected columns, along with a pointer to the corresponding row in the table.
- ▶ **Leftmost Prefix Rule:**
 - ▶ The index can efficiently be used for queries that filter on the leftmost column(s) in the index.
 - ▶ *Example: If index is on (A, B, C), queries on A, A+B, or A+B+C can use the index efficiently, but a query on only B or C cannot.*

Composite Index

- ▶ **Use Cases:**
 - ▶ When queries frequently filter on multiple columns together.
 - ▶ When sorting results by multiple columns.
 - ▶ Improves performance of WHERE, ORDER BY, and GROUP BY clauses involving multiple columns.

Composite index:

sql

```
CREATE INDEX idx_name ON EMPLOYEE(last_name, first_name);
```

- Efficient query:

sql

```
SELECT * FROM EMPLOYEE  
WHERE last_name = 'Smith' AND first_name = 'Carol';
```

- Also efficient:

sql

```
SELECT * FROM EMPLOYEE  
WHERE last_name = 'Smith';
```



Composite Index

- Not efficient:

sql

```
SELECT * FROM EMPLOYEE  
WHERE first_name = 'Carol';
```

Summary

Index Type	Key Feature	Notes
Primary	On primary key	Ordered
Secondary	On non-key column	Multiple allowed
Clustered	Physical order	Only one
Non-Clustered	Logical order	Many allowed
Dense	Entry for every record	Fastest search
Sparse	Entry per block	Less storage
Composite	Multi-column	Leftmost prefix rule



THANK YOU